

EX1

```
Set<Student> students = new HashSet<>();  
students.add(new Student(1, "ahmed"));  
students.add(new Student(1, "ahmed"));  
students.add(new Student(2, "osama"));  
students.add(new Student(2, "osama"));  
students.add(new Student(3, "ali"));
```

```
System.out.println(students);
```

السبب الرئيسي **HashSet**: يعتمد على دالتين مهمين جدًا عشان يكتشف التكرار **equals()** و **hashCode()**. وبما إن كلاس **Student** بتاعك ما عملتش **override** لهم، فجافا بتستخدم النسخة الافتراضية الموروثة من كلاس **Object** - وهي مش بتقارن بالقيم **(id, name)** ، وإنما بتقارن بـ مرجع الكائن في الذاكرة.

Object.hashCode(): بيرجع قيمة مبنية على مكان الكائن في الذاكرة (يعني كل كائن ليه قيمة مختلفة تقريبًا حتى لو نفس البيانات).

Object.equals(): بيقارن بـ **"reference equality"** ، يعني بيسأل: "هل الاتنين دول نفس الكائن بالظبط في الذاكرة؟" **(this == other)** فلما تعمل:

```
new Student(1, "ahmed")
```

```
new Student(1, "ahmed")
```

دول كائنين مختلفين في الذاكرة، حتى لو نفس القيم. فـ **HashSet**:

يبحسب الـ **hash code** لكل واحد فيهم → ببلاقيهم مختلفين (لأنهم كائنات مختلفة في الـ الميموري).
بالتالي بيحطهم في "**buckets**" مختلفة جوه الهاش سيت.
أصلاً مش بيوصل لمرحلة إنه يستخدم **equals()** عشان يقارن.
النتيجة: بيعتبرهم عنصرين مختلفين ويضيفهم الاتنين.
الحل: عمل **Override** لـ **equals()** و **hashCode()**

EX2

```
;Student student1 = new Student(1, "ahmed")  
;Student student2 = new Student(2, "osama")  
;Student student3 = new Student(1, "ali")  
  
;(<>)Set<Student> students = new HashSet  
;students.add(new Student(1, "ahmed"))  
;students.add(new Student(2, "osama"))  
  
;System.out.println(students.contains(student1))  
;System.out.println(students.contains(student2))  
;System.out.println(students.contains(student3))
```

السبب: الثلاثة رجعوا **false** لأنك على الأغلب عملت **override** لـ **equals()** بس من غير ما تعمل **override** لـ **hashCode()** وده بيوضح مبدأ مهم جداً هتحتاجه في الـ **research** بتاعك.

contains() في **HashSet** مش بتعمل مقارنة بـ **equals()** على طول. بالعكس، بتشتغل بخطوتين:

الخطوة الأولى: تحسب **hashCode()** للعنصر اللي بتدور عليه (زي **student1** ، وتستخدم القيمة دي عشان "تحدد" أنهي **bucket** (خانة) المفروض تدور فيها جوه الـ **HashSet**.
الخطوة الثانية: لو لقت عناصر في نفس الـ **bucket** ، هنا بس بتستخدم **equals()** عشان تقارن فعليًا.

المشكلة: لو **hashCode()** لسه بيرجع القيمة الافتراضية (المبنية على مكان الكائن في الذاكرة)، يبقى:

student1 (اللي انت عملته منفصل) ليه **hash code** مختلف تمامًا عن الكائن **new Student(1, "ahmed")** اللي فعلاً اتضاف جوه الـ **Set**.

فـ **HashSet** هيدور في **bucket** غلط أصلاً، ومش هيلقي فيه حاجة.

ومش هيوصل حتى لمرحلة إنه يستدعي **equals()** فيبيرجع **false** على طول، حتى لو البيانات متطابقة **100%**.

وده بيفسر ليه حتى **student1** اللي نفس بيانات العنصر المضاف بالظبط، **id=1** :
(**name="ahmed"** - رجع **false** لأن المقارنة بالـ **hash code** فشلت من الأول، قبل ما توصل لمرحلة مقارنة القيم).

الحل

لازم تعمل **override** للاتنين مع بعض

EX3

```
;Student student1 = new Student(1, "ahmed")  
;Student student2 = new Student(2, "osama")  
;Student student3 = new Student(1, "ali")  
  
;()<>HashMap<Student, Student> map = new HashMap  
map.put(new Student(1, "ahmed"), new Student(1,  
;"ahmed"))  
map.put(new Student(2, "osama"), new Student(2,  
;"osama"))  
  
;System.out.println(map.containsKey(student1))  
;System.out.println(map.containsKey(student2))  
;System.out.println(map.containsKey(student3))  
  
;System.out.println(map.containsValue(student1))  
;System.out.println(map.containsValue(student2))  
;System.out.println(map.containsValue(student3))
```

السبب هو نفس المشكلة اللي حصلت قبل كده بالظبط: أنت غالبًا عملت **override** لـ **equals()** بس من غير **hashCode()** بس هنا السؤال أعمق شوية، لأن **containsKey()** و

containsValue() بيشتغلوا بطريقة مختلفة تمامًا جوه الـ **HashMap** ، وده اللي خلاهم يتصرفوا بشكل مختلف.

ليه **containsValue** اشتغلت؟

containsValue() مش بتستخدم الـ **hashCode** خالص. هي بتعمل حاجة بسيطة: بتعمل بحث خطي - (**linear scan**) يعني بتمشي على كل الـ **values** واحد واحد جوه الماب، وتقارن كل واحد بـ **equals()** بس.

يعني:

لكل **value** جوه الماب:

لو **value.equals()** (اللي بندور عليه) → رجّع **true**

مفيش أي استخدام للـ **hash** هنا خالص، فيما إن (**equals()** شغالة صح) لأنك عملتلها (**override**)، ف **containsValue()** بتلاقي **student1** و **student2** عادي) لأن قيمهم مطابقة لقيم الـ **values** المضافة،) وبترجع **false** مع **student3** لأن مفيش **value** بنفس القيم.)

ليه **containsKey** فشلت؟

containsKey() بالعكس تمامًا - بتعتمد على الهاش زي ما شرحنا قبل كده:

تسبب **hashCode()** للـ **key** اللي بتدور عليه.

تستخدم القيمة دي عشان تحدد الـ **bucket** المفروض تدور فيه.

لو لقت **key** في نفس الـ **bucket** ، هنا بس تستخدم **equals()** للتأكد.

بما إن **hashCode()** لسه بالنسخة الافتراضية (مبنية على مكان الكائن في الذاكرة)، فـ **student1** (الكائن المنفصل اللي انت عملته) ليه **hash** مختلف تماماً عن الـ **key new Student(1, "ahmed")** اللي اتخط فعلاً جوه الماب. فـ **containsKey()** بتدور في **bucket** غلط من الأساس، ومش بتوصل حتى لمرحلة **equals()** → بترجع **false**.